MANUAL TÉCNICO Y DE USUARIO

JOBBRIDGE

Plataforma de Conexión Freelancer-PYME

Grupo #3

Kelvin Feng Wu

Heillyn Madriz Madrigal

Javier Núñez Sánchez

Desarrollo de Aplicaciones y Servicios Web

Sábado de 8:00 AM a 11:00 AM

Ing. Warner Carrillo Ramírez

I Cuatrimestre

Abril, 2026

TABLA DE CONTENIDO

1. INTRODUCCIÓN 5
2. PROPÓSITO DEL MANUAL 6
3. OBJETIVOS 7
3.1 Objetivo General 7
3.2 Objetivos Específicos 7
4. REQUISITOS DEL SISTEMA 8
4.1 Requisitos de Hardware 8
4.2 Requisitos de Software 8
4.3 Dependencias del Proyecto 9
5. ARQUITECTURA GENERAL DEL SISTEMA 10
5.1 Arquitectura General 10
5.2 Componentes del Frontend 11
5.3 Componentes del Backend 11
5.3.1 Capa de Configuración — config/ 12
5.3.2 Capa de Rutas — routes/ 12
5.3.3 Capa de Controladores — controllers/ 13
5.3.4 Capa de Modelos — models/ 14
5.3.5 Capa de Middleware — middleware/ 15
5.3.6 Seguridad del Backend 17
6. MANUAL TÉCNICO 18
6.1 Instalación del Editor de Código 18
6.2 Descarga del Proyecto 18
6.3 Instalación de Dependencias 19
Frontend (carpeta client) 19
Backend (carpeta server) 19
6.4 Configuración de Variables de Entorno 19
6.5 Ejecución del Proyecto 20
Iniciar Backend 20
Iniciar Frontend 20
6.6 Estructura Detallada del Proyecto 20

Configuración — config/db.js 20
Servidor Principal — server.js 21
6.7 Modelos de Datos 21
Modelo User — Decisiones de Diseño Clave 22
Modelo Contract — Máquina de Estados 22
6.8 Endpoints de la API REST 23
6.9 Middleware de Seguridad 24
6.10 Componentes Principales del Frontend 24
6.11 Explicación Detallada del Código 25
6.11.1 Punto de Entrada del Servidor — server.js 25
6.11.2 Conexión a Base de Datos — config/db.js 26
6.11.3 Middleware de Autenticación — auth.js 26
6.11.4 Middleware de Roles — role.js 26
6.11.5 Middleware de Archivos — upload.js 27
6.11.6 Controladores — Lógica de Negocio 27
6.11.7 Rutas y Protección de Endpoints 28
6.11.8 Frontend — Contextos Globales 29
6.11.9 Frontend — Servicio HTTP (api.js) 30
6.11.10 Frontend — Páginas Principales 31
7. MANUAL DE USUARIO 34
7.1 Acceso a la Plataforma 34
7.2 Registro de Usuarios 34
7.2.1 Registro como Freelancer 34
7.2.2 Registro como PYME 34
7.3 Inicio de Sesión 35
7.4 Panel Principal (Dashboard) 35
7.5 Gestión de Perfil 35
7.5.1 Perfil de Freelancer 35
7.5.2 Perfil de PYME 36
7.6 Publicación y Gestión de Servicios 36
7.6.1 Publicar un Servicio (Freelancer) 36
7.6.2 Explorar y Contratar un Servicio (PYME) 36
7.7 Sistema de Contratación 36
7.7.1 Proceso para PYME 36
7.7.2 Proceso para Freelancer 37
7.8 Sistema de Mensajería Interna 37

7.9 Calificaciones y Reseñas 37
7.10 Historial de Trabajos 38
8. CONCLUSIONES 39
9. RECOMENDACIONES 40
9.1 Recomendaciones Técnicas 40
9.2 Recomendaciones Funcionales 40
9.3 Recomendaciones de Seguridad 40
9.4 Recomendaciones de Experiencia de Usuario 41
10. BIBLIOGRAFÍA 42

1. INTRODUCCIÓN

En la actualidad, el desarrollo de plataformas web orientadas al empleo digital se ha convertido en una herramienta clave para conectar la oferta y la demanda de servicios de manera rápida, accesible y eficiente. Este fenómeno se enmarca en el crecimiento de la economía de plataformas y el trabajo digital, los cuales han transformado significativamente la forma en que se generan oportunidades laborales.

En este contexto, JobBridge surge como una solución tecnológica enfocada en facilitar la interacción entre freelancers y pequeñas y medianas empresas (PYMES), permitiendo la publicación, búsqueda, contratación y seguimiento de servicios especializados dentro de un entorno digital organizado y seguro. Esta propuesta responde también a problemáticas actuales como la dificultad de acceso al empleo, especialmente entre los jóvenes.

JobBridge integra funcionalidades esenciales para ambos tipos de usuarios: autenticación diferenciada por rol, administración de perfiles, publicación y visualización de servicios, sistema de contratación en línea, seguimiento del estado de proyectos, calificaciones bidireccionales y mensajería interna. Estas características construyen una experiencia funcional y dinámica, adaptada a las necesidades de profesionales independientes y de las empresas que requieren talento especializado.

Desde una perspectiva técnica, el sistema está estructurado bajo una arquitectura web moderna que separa claramente el backend (lógica de negocio y gestión de datos) del frontend (interacción con el usuario), siguiendo el modelo cliente-servidor ampliamente utilizado en el desarrollo de aplicaciones web modernas. El presente manual técnico y de usuario documenta de manera clara y comprensible el funcionamiento de JobBridge, facilitando su instalación, configuración y uso.

2. PROPÓSITO DEL MANUAL

El propósito del presente manual es ofrecer una guía completa, clara y estructurada que permita comprender, instalar, operar y mantener el sistema JobBridge en todas sus dimensiones técnicas y funcionales. Este documento está dirigido tanto a desarrolladores como a usuarios finales, proporcionando la información necesaria para garantizar un uso adecuado de la plataforma.

Desde el ámbito técnico, el manual detalla la organización del proyecto, sus componentes principales, la estructura del backend y frontend, y la interacción entre los diferentes módulos. Esto permite a los programadores y personal de soporte comprender la lógica que sustenta la plataforma, facilitando tareas de depuración, ampliación de funcionalidades y mantenimiento.

Desde la perspectiva del usuario, el documento describe el proceso de acceso, las funcionalidades disponibles para freelancers y PYMES, la forma de gestionar servicios, contratos, comunicaciones internas e historial de trabajos.

Audiencia del Manual

- Desarrolladores y equipo de soporte técnico
- Administradores de sistema
- Usuarios finales: Freelancers y PYMES
- Evaluadores y docentes del proyecto

3. OBJETIVOS

3.1 Objetivo General

Documentar el funcionamiento técnico y operativo de JobBridge, una plataforma web diseñada para la conexión entre jóvenes freelancers con pequeñas y medianas empresas, permitiendo la gestión eficiente de servicios, contratos, comunicación y seguimiento de proyectos dentro de un entorno digital accesible y seguro.

3.2 Objetivos Específicos

- Describir la arquitectura general del sistema, incluyendo la estructura del backend, frontend y la interacción entre sus componentes.
- Explicar el proceso de instalación, configuración y ejecución del proyecto, facilitando su mantenimiento y despliegue.
- Detallar las funcionalidades principales disponibles para freelancers y PYMES, incluyendo autenticación, perfiles, servicios, contratos, calificaciones y mensajería.
- Documentar el flujo lógico de cada módulo para el apoyo de futuras mejoras, ampliaciones o correcciones del sistema.
- Presentar las instrucciones de uso de la plataforma desde la perspectiva del usuario final, garantizando una experiencia clara y comprensible.
- Facilitar la comprensión del ciclo de vida de los proyectos dentro de JobBridge, desde la publicación del servicio hasta la finalización de la contratación.
- Proveer una guía de referencia estructurada que permita a desarrolladores y administradores el mantenimiento de la integridad y operatividad del sistema.

4. REQUISITOS DEL SISTEMA

4.1 Requisitos de Hardware

Componente	Especificación Mínima / Recomendada
Procesador	Mínimo: 2 núcleos (Intel/AMD) | Recomendado: 4 núcleos o superior
Memoria RAM	Mínimo: 4 GB | Recomendado: 8 GB o más
Espacio en disco	Mínimo: 10 GB | Recomendado: 20 GB
Resolución de pantalla	Mínimo: 1280 × 720 px | Recomendado: 1920 × 1080 px (Full HD)

4.2 Requisitos de Software

Componente	Detalle
Sistema Operativo	Windows 10/11, macOS 12+, Linux Ubuntu 20.04+
Node.js	Versión 16 o superior
Gestor de paquetes	NPM o Yarn
Base de datos	MongoDB (local o MongoDB Atlas)
Navegador web	Google Chrome, Firefox, Edge (versión actualizada)
Editor de código	Visual Studio Code (recomendado)

4.3 Dependencias del Proyecto

Capa	Dependencia	Función
Backend	Express	Framework servidor web
Backend	Mongoose	Modelado de datos MongoDB
Backend	Dotenv	Gestión de variables de entorno
Backend	Bcrypt	Encriptación de contraseñas
Backend	JsonWebToken	Gestión de sesiones (JWT)
Backend	Multer	Manejo de subida de archivos
Frontend	React 19	Interfaz de usuario (SPA)
Frontend	React Router	Navegación interna
Frontend	Axios	Consumo de API REST
Frontend	Vite	Bundler y servidor de desarrollo

5. ARQUITECTURA GENERAL DEL SISTEMA

El sistema JobBridge sigue una estructura monorepo con dos aplicaciones independientes dentro de un mismo repositorio: el servidor backend (Node.js + Express) y el cliente frontend (React + Vite). Ambas aplicaciones se comunican exclusivamente a través de la API REST. Esta separación favorece el mantenimiento, escalabilidad y reutilización del código, así como la independencia entre la interfaz de usuario y la lógica del sistema.

El flujo completo de una solicitud sigue esta cadena: la página React realiza una llamada HTTP mediante Axios, la cual llega a una ruta Express, pasa por los middlewares correspondientes, alcanza el controlador, interactúa con MongoDB mediante Mongoose, y finalmente devuelve una respuesta JSON que React utiliza para actualizar el estado de la interfaz.

5.1 Arquitectura General

Capa	Descripción
Frontend (Cliente)	Desarrollado con React 19 y Vite. Corre en el puerto 5173. Gestiona la interacción directa con el usuario mediante vistas dinámicas para registro, perfiles, servicios, contratos, historial y mensajería.
Backend (Servidor)	Implementado con Node.js y Express. Corre en el puerto 5000. Gestiona la lógica de negocio, seguridad y procesamiento de datos. Expone rutas y controladores para operaciones CRUD.
Base de Datos	MongoDB Atlas (NoSQL orientada a documentos). Almacena perfiles, proyectos, categorías, contratos y mensajes de forma flexible y escalable. Accedida mediante Mongoose.

5.2 Componentes del Frontend

El frontend integra los siguientes elementos organizados modularmente:

Componente	Descripción
🗁 pages/	Las 20 pantallas principales: Splash, Login, Register, Dashboard (PYME y Freelancer), Profile, PublicProfile, ServicesOffer, ServiceDetail, CreateModifyService, ServicesManage, NewContract, MyContracts, ContractDetails, MyRequests, Rating, Chat, Cart, PasswordRecovery.
🗁 components/	Elementos reutilizables: Layout (estructura de página con Sidebar), Sidebar (navegación diferenciada por rol, resalta página activa con useLocation()).
🗁 services/	api.js: instancia Axios con baseURL y request interceptor que adjunta JWT automáticamente. authService.js: funciones loginUser() y registerUser() como wrappers de Axios.
🗁 context/	AuthContext: gestiona user, token, login(), logout(), updateUser() con persistencia en localStorage. CartContext: gestiona carrito de servicios con addToCart(), removeFromCart(), getTotal().
🗁 assets/	Imágenes, íconos, logos y hoja de estilos global (index.css) con todas las clases CSS (.panel, .btn, .card, .inline, etc.).

5.3 Componentes del Backend

El backend de JobBridge está construido sobre Node.js con Express como framework servidor, y sigue una arquitectura en capas que separa claramente las responsabilidades de cada módulo. A continuación, se describe en detalle cada uno de sus componentes:

5.3.1 Capa de Configuración — config/

Esta capa es el punto de arranque del sistema. Contiene los archivos responsables de inicializar los servicios fundamentales antes de que el servidor comience a recibir solicitudes.

Archivo	Responsabilidad
db.js	Establece la conexión con MongoDB mediante mongoose.connect(). Si la conexión falla, el proceso se detiene con process.exit(1) para evitar ejecutar el servidor sin base de datos.
server.js	Archivo de entrada principal. Carga dotenv, inicializa Express, registra middleware global (cors, express.json), monta todas las rutas y levanta el servidor solo después de conectar a MongoDB.

Flujo de inicialización del servidor

1. dotenv carga las variables del archivo .env (PORT, MONGO_URI, JWT_SECRET).
2. Se configura Express con los middlewares globales: cors() y express.json().
3. Se importan y montan todos los módulos de rutas bajo el prefijo /api/.
4. connectDB() establece la conexión con MongoDB de forma asíncrona.
5. Solo si la conexión es exitosa, app.listen() inicia el servidor en el puerto configurado.

IMPORTANTE: MONGO_URI debe ser la ÚLTIMA línea del archivo .env. La cadena de conexión de MongoDB Atlas es muy larga y puede romper el parser de dotenv si hay variables definidas después de ella.

5.3.2 Capa de Rutas — routes/

Las rutas definen los puntos de entrada de la API REST. Cada archivo de ruta corresponde a una entidad del sistema y delega el procesamiento al controlador correspondiente. Todas las rutas están montadas bajo el prefijo /api/.

Archivo de Ruta	Prefijo	Endpoints principales
authRoute.js	/api/auth	POST /register, POST /login
userRoute.js	/api/users	GET /me, PUT /me, POST /me/picture, GET /:id
serviceRoute.js	/api/services	GET /, POST /, GET /:id, PUT /:id, DELETE /:id, POST /:id/images
contractRoute.js	/api/contracts	GET /, POST /, GET /:id, PUT /:id/status, PUT /:id/milestones
ratingRoute.js	/api/ratings	POST /, GET /user/:userId, GET /service/:serviceId
messageRoute.js	/api/messages	POST /, GET /conversations, GET /:contractId

Las rutas protegidas utilizan el middleware auth antes de llegar al controlador. Las rutas que requieren un rol específico también aplican role(). El orden de definición importa: /conversations debe definirse antes de /:contractId para que Express no interprete "conversations" como un parámetro de URL.

5.3.3 Capa de Controladores — controllers/

Los controladores son el núcleo de la lógica de negocio. Reciben las solicitudes HTTP desde las rutas, interactúan con los modelos de datos, aplican las reglas del sistema y devuelven respuestas estructuradas en formato JSON.

Controlador	Funciones Principales
authController.js	register(): valida contraseña con 5 regex (longitud, mayúscula, minúscula, número, carácter especial), verifica correo único, cifra con bcrypt.genSalt(10) + bcrypt.hash(), genera JWT con 24h de expiración. login(): autentica con bcrypt.compare(), retorna el mismo mensaje genérico para usuario no encontrado y contraseña incorrecta (evita enumeración de cuentas).
userController.js	getProfile(): retorna perfil con .select("-passwordHash") para nunca enviar la contraseña al frontend. updateProfile(): field whitelisting, solo campos en allowedFields pueden modificarse, previniendo escalación de privilegios (cambio de rol, email).

	uploadProfilePicture(): almacena la ruta relativa /uploads/filename.jpg, no la ruta absoluta del sistema.
serviceController.js	getAll(): construye filtro dinámico con operadores MongoDB $lte/$gte para precio y entrega. Para sort por calificación usa Promise.all() para calcular promedios en paralelo. createService(): vincula freelancerId desde req.user.id (JWT), nunca desde el body. deleteService(): compara freelancerId.toString() con req.user.id porque ObjectId ≠ string sin conversión.
contractController.js	createContract(): calcula platformFee = price * 0.05 en el servidor (nunca confiar en el cliente para cálculos financieros). updateContractStatus(): implementa máquina de estados con objeto allowedTransitions por rol y estado. Registra startDate automáticamente al aceptar y endDate al completar.
ratingController.js	createRating(): valida que el contrato esté completado, que el usuario sea parte del contrato, y que no exista calificación previa (un rating por usuario por contrato). Determina toUserId automáticamente según el rol del emisor.
messageController.js	getMessages(): usa updateMany() para marcar como read: true todos los mensajes no leídos del usuario en una sola operación de base de datos. getMyConversations(): para cada contrato obtiene el último mensaje (.sort({createdAt: -1})) y el conteo de no leídos (countDocuments).

5.3.4 Capa de Modelos — models/

Los modelos definen la estructura de los documentos almacenados en MongoDB mediante esquemas de Mongoose. Cada modelo establece los campos, sus tipos, validaciones, valores por defecto y las relaciones entre entidades a través de referencias ObjectId.

Modelo	Entidad y decisiones de diseño
User.js	Fusiona las entidades relacionales (Usuario, Freelancer, Cliente, Empresa) en un único documento con campo role: enum["freelancer","pyme"]. company es subdocumento embebido (datos de empresa de la PYME). skills es array de strings. timestamps:true añade createdAt y updatedAt automáticamente.
Service.js	freelancerId referencia User con ref:"User". 15 categorías predefinidas con enum. min:0 en price previene precios negativos. status con 4 estados posibles. images es array de rutas de archivos.
Contract.js	Almacena freelancerId y clientId directamente (sin derivar de Service) para acelerar consultas. price, platformFee y total son snapshots del momento de contratación. Implementa máquina de estados: requested → accepted → in_progress → completed → cancelled. milestones es array de subdocumentos embebidos.
Rating.js	Sistema bidireccional: fromUserId (quien califica) y toUserId (quien recibe). contractId previene calificaciones duplicadas. score con min:1, max:5 validado a nivel de esquema.
Message.js	Mensajes organizados por contrato (contractId), no por par de usuarios. read:false por defecto. El campo timestamps proporciona el orden cronológico de los mensajes.

5.3.5 Capa de Middleware — middleware/

El middleware actúa como capa intermedia entre la solicitud HTTP y el controlador. Su función es interceptar, validar o transformar las solicitudes antes de que lleguen a la lógica de negocio, garantizando seguridad y consistencia en toda la API.

Middleware	Funcionamiento Detallado
auth.js	Extrae el JWT del encabezado Authorization usando optional chaining (?.) para evitar crash si no existe. jwt.verify() valida firma con JWT_SECRET Y expiración en un solo paso. Adjunta el payload decodificado a req.user. Responde con HTTP 401 con mensaje genérico "Invalid token" sin revelar si expiró o fue manipulado.
role.js	Higher-order function: role(...allowedRoles) retorna un middleware. Usa rest parameters (...) para aceptar múltiples roles. Siempre corre después de auth (que establece req.user). HTTP 403 significa "autenticado pero sin permiso", diferente al 401 de auth.
upload.js	Multer con diskStorage: filename genera nombre único con Date.now() + Math.random() para evitar colisiones. fileFilter valida MIME type (jpeg, png, gif, webp) rechazando ejecutables o scripts. limits.fileSize = 5MB. upload.single("picture") para perfiles, upload.array("images", 5) para servicios.

Flujo de una solicitud HTTP protegida

Cliente → [auth.js: verifica JWT] → [role.js: verifica rol]

→ [Controller: lógica de negocio] → [Model: consulta MongoDB]

→ Respuesta JSON al cliente

Si auth.js falla: HTTP 401 — "No token" o "Invalid token"

Si role.js falla: HTTP 403 — "Access denied for your role"

Si el controller falla: HTTP 400/404/500 con mensaje descriptivo

5.3.6 Seguridad del Backend

Mecanismo de Seguridad	Implementación
JWT + bcrypt	Tokens firmados con JWT_SECRET, expiración 24h. Contraseñas hasheadas con bcrypt (10 rounds = 2^10 iteraciones), nunca almacenadas en texto plano.
Control de acceso por rol	role() middleware impide acceso a rutas restringidas. Field whitelisting en update controllers previene escalación de privilegios.
CORS configurado	cors() controla los orígenes permitidos, previniendo solicitudes desde dominios no autorizados.
Variables de entorno	JWT_SECRET y MONGO_URI en .env, excluido del repositorio con .gitignore.
Validación de entrada	enum en esquemas Mongoose rechaza valores inválidos antes de llegar a la BD. min/max en campos numéricos. fileFilter en uploads.
Mensajes de error genéricos	"Invalid credentials" en login (sin indicar si el email existe). "Invalid token" sin distinguir expirado/manipulado. Previene ataques de enumeración.

6. MANUAL TÉCNICO

6.1 Instalación del Editor de Código

Para el desarrollo y mantenimiento de JobBridge se recomienda Visual Studio Code, disponible en https://code.visualstudio.com. Una vez instalado, se recomienda agregar las extensiones: ESLint, Prettier, MongoDB for VS Code y GitLens para mayor productividad.

6.2 Descarga del Proyecto

Una vez descargado el proyecto en formato .zip, se debe descomprimir y arrastrar la carpeta al editor de código. El proyecto sigue una estructura monorepo con dos carpetas principales: client/ (frontend React) y server/ (backend Node.js).

Estructura del Proyecto

```
JobBridge/
  ├── server/        → Node.js + Express REST API (backend)
  │   ├── config/     → Conexión a base de datos
  │   ├── models/     → Esquemas Mongoose
  │   ├── controllers/ → Lógica de negocio
  │   ├── middleware/  → auth, role, upload
  │   ├── routes/     → Definición de endpoints
  │   └── server.js   → Punto de entrada
  ├── client/        → React 19 + Vite SPA (frontend)
  │   └── src/
  │       ├── components/ → Layout, Sidebar
  │       ├── context/    → AuthContext, CartContext
  │       ├── services/   → api.js, authService.js
  │       └── pages/      → 19 páginas
  └── .env           → Variables de entorno (secretos)
```

6.3 Instalación de Dependencias

Se deben instalar las dependencias tanto del frontend como del backend. Abrir dos terminales en Visual Studio Code (Terminal → New Terminal) y ejecutar:

Frontend (carpeta client)

```
cd client
npm install
```

Backend (carpeta server)

```
cd server
npm install
```

Para verificar que las dependencias se instalaron correctamente, confirmar la existencia de la carpeta node_modules/ dentro de cada directorio respectivo.

6.4 Configuración de Variables de Entorno

En la raíz del proyecto se debe crear o verificar el archivo .env con las siguientes variables. Es crítico que MONGO_URI sea la última línea del archivo para evitar errores de parseo de dotenv con la cadena de conexión de MongoDB Atlas.

```
PORT=5000
JWT_SECRET=jobbridge_secret_key_2026
MONGO_URI=mongodb+srv://usuario:password@cluster.mongodb.net/JobBridgeDB
```

Variables de Entorno — Descripción

- PORT: Puerto donde corre el servidor Express (por defecto 5000).
- JWT_SECRET: Clave para firmar y verificar tokens JWT. Debe ser larga y aleatoria.
- MONGO_URI: Cadena de conexión a MongoDB Atlas. Debe ser la ÚLTIMA línea del archivo.

IMPORTANTE: Incluir .env en .gitignore para nunca exponer credenciales en el repositorio.

6.5 Ejecución del Proyecto

Para ejecutar el proyecto se deben correr ambos servicios de forma simultánea en terminales separadas:

Iniciar Backend

```
cd server
npm run dev
# Salida esperada:
# MongoDB connected
# Server running on port 5000
```

Iniciar Frontend

```
cd client
npm run dev
# Salida esperada:
# VITE v4.x.x  ready in 300 ms
# ➜  Local:   http://localhost:5173/
```

Una vez ambos servicios están corriendo, abrir el enlace local que aparece en la terminal del cliente (http://localhost:5173) para acceder a la aplicación. El servidor no arranca si MongoDB no conecta exitosamente.

6.6 Estructura Detallada del Proyecto

Configuración — config/db.js

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connected");
  } catch (error) {
    console.error("MongoDB connection error:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Archivo crítico de una sola responsabilidad: conectar a MongoDB. La separación es intencional, server.js no necesita saber los detalles de la conexión, solo llama a connectDB(). Si la conexión falla, process.exit(1) detiene el proceso evitando un servidor sin base de datos.

Servidor Principal — server.js

```
require("dotenv").config({ path: "../.env" });
const express = require("express");
const cors = require("cors");
const connectDB = require("./config/db");
const path = require("path");

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors());           // Permite peticiones desde localhost:5173
app.use(express.json());   // Parsea JSON en req.body

app.use("/api/auth",      require("./routes/authRoute"));
app.use("/api/users",     require("./routes/userRoute"));
app.use("/api/services",  require("./routes/serviceRoute"));
app.use("/api/contracts", require("./routes/contractRoute"));
app.use("/api/ratings",   require("./routes/ratingRoute"));
app.use("/api/messages",  require("./routes/messageRoute"));

app.use("/uploads", express.static(path.join(__dirname, "uploads")));

connectDB().then(() => {
  app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
});
```

app.use("/api/auth", authRoutes) significa que cualquier solicitud con prefijo /api/auth será manejada por el router de autenticación. Dentro de ese router, router.post("/register") se convierte en POST /api/auth/register desde el exterior. La línea de uploads sirve los archivos subidos como contenido estático accesible por URL. El servidor solo se levanta después de confirmar la conexión a MongoDB.

6.7 Modelos de Datos

Los modelos son la representación de las entidades del sistema en MongoDB. Mongoose agrega validaciones a nivel de aplicación sobre una base de datos que es intrínsecamente sin esquema.

Modelo User — Decisiones de Diseño Clave

```
email: { type: String, required: true, unique: true,
         lowercase: true, trim: true },
// unique:true crea índice MongoDB que rechaza duplicados a nivel de BD
// lowercase convierte "JAVIER@test.com" → "javier@test.com" automáticamente

passwordHash: { type: String, required: true },
// Almacena HASH, nunca la contraseña. Nombre explícito para claridad.

role: { type: String, enum: ["freelancer", "pyme"], required: true },
// Fusiona 4 tablas relacionales en 1 documento. role discrimina el tipo.

company: { name: String, cedulaJuridica: String, address: String,
           industry: String, size: String },
// Subdocumento embebido — en SQL sería tabla Empresa con FK.
// Se embebe porque siempre se accede junto con el usuario.

skills: [{ type: String, trim: true }],
// Array nativo — en SQL necesitaría tabla intermedia FreelancerSkills.

}, { timestamps: true });
// Añade createdAt y updatedAt automáticamente en cada save().
```

Modelo Contract — Máquina de Estados

```
status: {
  type: String,
  enum: ["requested","accepted","in_progress","completed","cancelled"],
  default: "requested"
},
// Estado del ciclo de vida. El controlador valida transiciones válidas.

price:       { type: Number, required: true },
platformFee: { type: Number, default: 0 },
total:       { type: Number, required: true },
// Snapshots del precio en el momento de creación.
// Si el freelancer sube su precio, los contratos existentes no cambian.

milestones: [{
  title: String,
  status: { type: String, enum: ["pending","in_progress","completed"] },
  completedAt: Date
}]
// Array de subdocumentos embebidos — seguimiento de hitos del proyecto.
```

6.8 Endpoints de la API REST

Método	Endpoint	Descripción
POST	/api/auth/register	Registrar nuevo usuario
POST	/api/auth/login	Iniciar sesión — retorna JWT
GET	/api/users/me	Perfil del usuario autenticado
PUT	/api/users/me	Actualizar perfil (field whitelisting)
POST	/api/users/me/picture	Subir foto de perfil (multipart)
GET	/api/users/:id	Perfil público de otro usuario
GET	/api/services	Listar servicios (filtros query params)
POST	/api/services	Crear servicio (solo freelancer)
GET	/api/services/:id	Detalle de un servicio
PUT	/api/services/:id	Editar servicio (solo dueño)
DELETE	/api/services/:id	Eliminar servicio (solo dueño)
POST	/api/services/:id/images	Subir imágenes al servicio
GET	/api/contracts	Contratos del usuario autenticado
POST	/api/contracts	Crear contrato (solo PYME)
GET	/api/contracts/:id	Detalle de un contrato
PUT	/api/contracts/:id/status	Cambiar estado (máquina de estados)
PUT	/api/contracts/:id/milestones	Actualizar hitos (solo freelancer)
POST	/api/ratings	Crear calificación
GET	/api/ratings/user/:userId	Calificaciones de un usuario
GET	/api/ratings/service/:serviceId	Calificaciones de un servicio
POST	/api/messages	Enviar mensaje
GET	/api/messages/conversations	Lista de conversaciones

GET	/api/messages/:contractId	Mensajes de un contrato

6.9 Middleware de Seguridad

Middleware	Función
auth.js	Verifica JWT en encabezado Authorization. Rechaza con HTTP 401 si falta o es inválido. Adjunta payload decodificado a req.user.
role.js	Higher-order function role(...roles). Verifica req.user.role. HTTP 403 si el rol no está permitido.
upload.js	Multer con diskStorage. Filtra MIME types, genera nombres únicos, limita a 5MB.

6.10 Componentes Principales del Frontend

Componente / Página	Función Principal
Splash.jsx	Pantalla de bienvenida. useEffect con setTimeout(3000) redirige a login. Cleanup con clearInterval previene memory leaks.
Register.jsx	Formulario con 5 validaciones de contraseña client-side. El servidor valida independientemente (defensa en profundidad). Radio buttons estilizados para selección de rol.
Login.jsx	e.preventDefault() evita recarga. login() de AuthContext guarda en estado + localStorage. Mensaje de error con optional chaining: err.response?.data?.message.
DashboardPYME.jsx	Fetch único de contratos, luego filtra client-side por status para derivar estadísticas. Más eficiente que 3 llamadas separadas.
DashboardFreelancer.jsx	Promise.all() para fetchear servicios y contratos en paralelo. Calcula ganancias con reduce() y tasa de completación.

ServicesOffer.jsx	Estado local por cada filtro. setTimeout(..., 0) en handleClear difiere el fetch al siguiente tick asegurando que los resets se aplicaron.
CreateModifyService.jsx	Componente dual create/edit. isEditing = Boolean(useParams().id). FormData para upload de imágenes con Content-Type: multipart/form-data.
ContractDetails.jsx	Botones de acción renderizados condicionalmente por rol + estado actual. Espeja la lógica allowedTransitions del backend en el frontend.
Chat.jsx	Polling HTTP con setInterval(fetchMessages, 5000). messagesEndRef con scrollIntoView para auto-scroll. Alineación de mensajes por senderId._id vs user.id.
Profile.jsx	Toggle view/edit con estado editing. updateUser() de AuthContext actualiza Sidebar inmediatamente al cambiar foto de perfil sin recarga.

6.11 Explicación Detallada del Código

6.11.1 Punto de Entrada del Servidor — server.js

El archivo server.js realiza cinco operaciones en orden estrictamente definido. Primero carga las variables de entorno con require("dotenv").config({ path: "../.env" }), donde la ruta "../.env" es necesaria porque el servidor corre desde la carpeta server/ pero el archivo .env vive en la raíz del proyecto. Segundo, registra los dos middlewares globales que procesan TODAS las solicitudes entrantes: cors() agrega el encabezado Access-Control-Allow-Origin para permitir peticiones desde el puerto 5173 de React, y express.json() parsea el cuerpo JSON y lo coloca en req.body. Sin este último, req.body sería undefined en todos los controladores. Tercero, monta los seis módulos de rutas bajo sus prefijos respectivos. Cuarto, expone la carpeta uploads/ como archivos estáticos mediante express.static(), haciendo accesibles las imágenes subidas vía URL. Quinto y más importante, llama a connectDB() y solo dentro del callback .then() levanta el servidor con app.listen(). Esta secuencia garantiza que el servidor nunca acepte solicitudes sin base de datos disponible.

6.11.2 Conexión a Base de Datos — config/db.js

Este archivo encapsula un único concepto: establecer la conexión con MongoDB Atlas. La función connectDB es asíncrona y utiliza async/await para que el código asíncrono de Mongoose se lea como código síncrono. mongoose.connect() recibe la cadena de conexión desde process.env.MONGO_URI, que dotenv inyectó al inicio. El bloque try/catch captura dos tipos de fallo: errores de red (Atlas inaccesible) y errores de autenticación (credenciales incorrectas). En ambos casos, process.exit(1) termina el proceso Node.js con código de error, lo que es preferible a ejecutar un servidor que responderá con errores de base de datos en cada endpoint. La separación en un archivo propio sigue el principio de responsabilidad única: si en el futuro se necesita agregar configuración de connection pooling, retry logic o múltiples bases de datos, solo este archivo necesita modificarse.

6.11.3 Middleware de Autenticación — auth.js

El middleware de autenticación sigue un patrón de guardas secuenciales. Primero extrae el token del encabezado Authorization usando optional chaining (?.) que previene un TypeError si el encabezado no existe, retornando undefined en su lugar. El método .replace("Bearer ", "") elimina el prefijo estándar dejando solo el token. Si no hay token, retorna inmediatamente con HTTP 401, el return es crítico para evitar que la ejecución continúe hacia next(). Luego jwt.verify() realiza dos verificaciones en una sola llamada: confirma que la firma del token coincide con JWT_SECRET (asegurando que no fue manipulado) y que el token no ha expirado (la expiración se embebe en el payload durante la creación). Si es válido, el payload decodificado { id, role } se adjunta a req.user, disponible para todos los middleware y controladores posteriores en la cadena. Cualquier fallo de verificación lanza una excepción capturada por el catch, que responde con el mensaje genérico "Invalid token" sin distinguir entre token expirado, firma inválida o formato incorrecto, lo cual es una práctica de seguridad deliberada para no revelar información a posibles atacantes.

6.11.4 Middleware de Roles — role.js

El middleware de roles es una higher-order function, una función que retorna otra función. Esta técnica permite parametrizar el middleware: role("pyme") crea un middleware que solo permite PYMEs, mientras que role("freelancer", "pyme") permitiría ambos roles. El parámetro rest (...allowedRoles) recoge todos los argumentos en un array, por lo que la función puede recibir cualquier número de roles. La función retornada accede a req.user.role, que el middleware auth.js adjuntó previamente, razón por la que role siempre debe ir después de auth en la cadena. La diferencia entre HTTP 401 (auth fallido) y HTTP 403 (role fallido) es semánticamente importante: 401 significa "no te conozco", mientras que 403 significa "te conozco pero no tienes permiso".

6.11.5 Middleware de Archivos — upload.js

Multer maneja las solicitudes multipart/form-data, el formato estándar para upload de archivos en HTTP. La configuración diskStorage define dos aspectos del almacenamiento: destination especifica la carpeta uploads/ como destino, y filename genera un nombre único combinando Date.now() (timestamp en milisegundos) con Math.round(Math.random() * 1E9) (número aleatorio de 9 dígitos), más la extensión original del archivo con path.extname(). Esta combinación hace prácticamente imposible que dos archivos tengan el mismo nombre. La función fileFilter es una capa de seguridad esencial: verifica el MIME type del archivo contra una lista blanca de imágenes permitidas, rechazando cualquier otro tipo de archivo. Sin este filtro, un atacante podría subir archivos ejecutables al servidor. El límite de 5MB (5 × 1024 × 1024 bytes) previene ataques de denegación de servicio por archivos enormes.

6.11.6 Controladores — Lógica de Negocio

Los controladores de JobBridge implementan varios patrones de seguridad y diseño que vale la pena entender en detalle.

authController.js — Registro y Login

REGISTRO: Las 5 validaciones de contraseña usan expresiones regulares independientes para dar mensajes de error específicos. bcrypt.genSalt(10) genera un salt aleatorio con 2^10 = 1024 iteraciones de hashing (estándar de la industria para balance seguridad/rendimiento). El token JWT se crea con payload mínimo { id, role } — solo lo necesario para autenticar solicitudes futuras.

LOGIN: Usa el mismo mensaje genérico "Invalid credentials" tanto para usuario no encontrado como para contraseña incorrecta. Si dijera "email no registrado" vs "contraseña incorrecta", un atacante podría enumerar qué emails existen en el sistema.

serviceController.js — Filtros y Consultas Avanzadas

El método getAll() construye un objeto filter dinámico que comienza con { status: "active" } y agrega condiciones según los query params recibidos. Los operadores MongoDB $lte (less than or equal) y $gte (greater than or equal)

permiten filtros de rango sobre el precio y días de entrega.

Para ordenamiento por calificación (sort=rating_desc), el controlador usa Promise.all() para calcular el promedio de cada servicio en paralelo, más eficiente que esperar cada cálculo secuencialmente. .toObject() convierte el documento Mongoose a un objeto JavaScript plano para poder agregar la propiedad avgRating.

En deleteService(), la comparación freelancerId.toString() !== req.user.id es necesaria porque freelancerId es un ObjectId (tipo especial de MongoDB) mientras req.user.id es un string, sin .toString(), === siempre sería false aunque sean el mismo ID, creando una vulnerabilidad donde nadie podría eliminar sus propios servicios.

contractController.js — Máquina de Estados y Cálculos Financieros

El platformFee se calcula en el servidor: service.price * 0.05.
NUNCA se confía en cálculos financieros enviados por el cliente, podrían manipularse. El total = price + platformFee se almacena como snapshot para que cambios futuros de precio no afecten contratos existentes.

allowedTransitions define exactamente qué cambios de estado puede hacer cada rol:
PYME: puede cancelar desde "requested" o "in_progress" solamente.
Freelancer: puede aceptar/rechazar desde "requested", iniciar desde "accepted", y completar desde "in_progress".

Los timestamps startDate y endDate se registran automáticamente al cambiar de estado, sin necesidad de que el frontend los envíe.

6.11.7 Rutas y Protección de Endpoints

```
// authRoute.js — SIN middleware (endpoints públicos)
router.post("/register", register);
router.post("/login", login);

// serviceRoute.js — Mix público/privado
router.get("/", getAll);                                    // PÚBLICO
```

```
router.get("/:id", getById);                                // PÚBLICO
router.post("/", auth, role("freelancer"), createService); // PRIVADO
router.put("/:id", auth, role("freelancer"), updateService);
router.delete("/:id", auth, role("freelancer"), deleteService);
router.post("/:id/images", auth, role("freelancer"),
            upload.array("images", 5), uploadServiceImages);
// ^ Cadena de 4 middleware: auth → role → upload → controller

// contractRoute.js — Solo PYMEs crean, ambos ven
router.post("/", auth, role("pyme"), createContract);
router.get("/", auth, getMyContracts);
router.put("/:id/status", auth, updateContractStatus);

// messageRoute.js — ORDEN IMPORTA
router.get("/conversations", auth, getMyConversations); // PRIMERO
router.get("/:contractId", auth, getMessages);          // DESPUÉS
// Si fuera al revés, "conversations" matchearía como :contractId
```

El catálogo de servicios (GET /api/services) y el detalle de servicio (GET /api/services/:id) son públicos intencionalmente, cualquiera puede explorar los servicios sin estar registrado. Las calificaciones y perfiles públicos también son públicos para aumentar la visibilidad de los freelancers.

6.11.8 Frontend — Contextos Globales

AuthContext — Persistencia de Sesión

getStoredUser() lee localStorage al inicializar. La sintaxis useState(getStoredUser) sin paréntesis es "lazy initialization", React llama la función solo una vez en el primer render, no en cada re-render. Con paréntesis (getStoredUser()) se ejecutaría en cada render.

login() guarda en estado React (para UI inmediata) Y en localStorage (para persistencia en recargas). logout() limpia ambos, cuando user se vuelve null, todos los route guards en App.jsx re-renderizan y redirigen a login.

updateUser({ profilePicture: newPath }) usa spread: { ...user, ...updatedFields } para actualizar solo los campos cambiados, preservando el resto del objeto user.

CartContext — Inmutabilidad de Estado React

addToCart: verifica duplicados con .find() antes de agregar.

setCart([...cart, service]), el spread crea un NUEVO array.

NUNCA hacer cart.push(service), mutaría el array existente y React

no detectaría el cambio, por lo que no habría re-render.

removeFromCart: .filter() también crea un nuevo array.

getTotal: .reduce() acumula la suma de precios.

Retorna { subtotal, fee: subtotal*0.05, total }, mismo cálculo

que el servidor, para mostrar el desglose al usuario antes de confirmar.

6.11.9 Frontend — Servicio HTTP (api.js)

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:5000/api"
});

// Request interceptor — corre ANTES de cada solicitud
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

Este archivo es la pieza central de comunicación frontend-backend. axios.create() produce una instancia con baseURL preconfigurada, api.get("/services") se convierte automáticamente en GET http://localhost:5000/api/services. El request interceptor es el patrón clave: corre automáticamente antes de CADA solicitud y adjunta el JWT del localStorage al encabezado Authorization. Sin esto, cada página tendría que manejar manualmente la autenticación. Gracias a este interceptor, toda la app solo necesita importar api y llamar api.get(), api.post(), etc., sin preocuparse por los headers de autenticación.

6.11.10 Frontend — Páginas Principales

A continuación, se documentan los patrones y decisiones técnicas más importantes de las páginas de la aplicación.

App.jsx — Sistema de Rutas con Guards

getDashboardRoute() retorna la URL correcta según user.role.

Route guards con operador ternario:

- Rutas públicas: !user ? <Splash/> : <Navigate to={getDashboardRoute()}/>
  → Si NO está logueado, muestra la página. Si SÍ, redirige al dashboard.
- Rutas protegidas: user ? <Dashboard/> : <Navigate to="/login"/>
  → Si SÍ está logueado, muestra la página. Si NO, redirige a login.
- Rutas por rol: user && user.role=="pyme" ? <DashboardPYME/> : <Navigate/>
  → Requiere estar logueado Y tener el rol correcto.

Catch-all route path="*": previene páginas 404 en blanco.

:id en las rutas es un URL parameter capturado con useParams().

Dashboard Pages — Optimizaciones de Fetch

DashboardPYME: Un fetch de contratos, filtra client-side.

```
const active = contracts.filter(c => c.status === "in_progress")
```

Más eficiente que 3 llamadas API separadas.

DashboardFreelancer: Promise.all() para paralelizar fetches.

```
const [servicesRes, contractsRes] = await Promise.all([
  api.get("/services"),
  api.get("/contracts")
]);
```

Envía ambas solicitudes simultáneamente, tiempo total = max(t1, t2)

en lugar de t1 + t2 con solicitudes secuenciales.

Chat.jsx — Polling y Auto-scroll

HTTP Polling con setInterval:

```
  const interval = setInterval(fetchMessages, 5000);
  return () => clearInterval(interval);  // cleanup
```

El cleanup previene memory leaks: cuando el usuario cambia de contrato o desmonta el componente, clearInterval cancela el intervalo pendiente.

Auto-scroll con useRef:

```
  messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
```

Un vacío al final de la lista de mensajes. scrollIntoView lo hace visible, efectivamente scrolleando al último mensaje.

Alineación de mensajes, maneja dos casos del senderId:

```
  msg.senderId?._id === user?.id  // cuando está populated (objeto)
  || msg.senderId === user?.id    // cuando es solo el ObjectId string
```

ContractDetails.jsx — Buttons Condicionales

```
{isFreelancer && contract.status === "requested" && (
  <><button onClick={() => update("accepted")}>Aceptar</button>
     <button onClick={() => update("cancelled")}>Rechazar</button></>
)}
{isFreelancer && contract.status === "accepted" && (
  <button onClick={() => update("in_progress")}>Iniciar trabajo</button>
)}
{isFreelancer && contract.status === "in_progress" && (
  <button onClick={() => update("completed")}>Marcar completado</button>
)}
```

Espeja la lógica allowedTransitions del backend, el UI solo muestra acciones válidas, pero el backend SIEMPRE valida independientemente.

NewContract.jsx — Cálculo de Fechas y Formateo

Cálculo de advertencia de deadline:

```
const diffDays = Math.ceil((deadline - today) / (1000*60*60*24));
// Diferencia en ms → dividir por ms-por-día → redondear hacia arriba
```

Formateo en tiempo real de número de tarjeta:

```
value = value.replace(/[^0-9]/g, "");  // solo dígitos
value = value.substring(0, 16);        // máx 16
value = value.replace/(.{4})/g, "$1 ").trim(); // espacio cada 4
// Produce: "1234 5678 9012 3456" mientras el usuario escribe
```

7. MANUAL DE USUARIO

El presente Manual de Usuario describe de manera clara y detallada los pasos necesarios para utilizar la plataforma JobBridge. Las instrucciones aplican tanto para Freelancers como para PYMES, indicando en cada caso las opciones específicas disponibles según el rol.

7.1 Acceso a la Plataforma

Luego de ejecutar el proyecto con npm run dev en ambas terminales (client y server), abrir el enlace local que aparece en la terminal del cliente (http://localhost:5173). El sistema mostrará la pantalla de bienvenida (Splash) y redirigirá automáticamente a la página de inicio de sesión tras 3 segundos.

7.2 Registro de Usuarios

7.2.1 Registro como Freelancer

1. Hacer clic en "Crear cuenta" en la pantalla de inicio de sesión.
2. Seleccionar la opción "Freelancer" en el selector de rol.
3. Completar nombre completo, correo electrónico y contraseña segura (mínimo 8 caracteres con mayúscula, minúscula, número y carácter especial).
4. Confirmar contraseña y presionar "Crear cuenta".
5. El sistema creará la cuenta y redirigirá al dashboard del freelancer.

7.2.2 Registro como PYME

6. Hacer clic en "Crear cuenta" y seleccionar la opción "PYME".
7. Completar nombre, correo empresarial y contraseña segura.
8. Tras el registro, completar el perfil con datos de la empresa: nombre comercial, cédula jurídica, dirección, industria y tamaño.
9. Presionar "Guardar" para confirmar.

7.3 Inicio de Sesión

10. Ingresar el correo electrónico registrado.
11. Ingresar la contraseña.
12. Hacer clic en "Ingresar".
13. Si las credenciales son correctas, el sistema redirigirá al panel principal según el rol del usuario.
14. Si los datos son incorrectos, se mostrará un mensaje de error.

7.4 Panel Principal (Dashboard)

Una vez autenticado, el usuario accede al panel principal con acceso a todas las funciones de la plataforma según su rol:

Para Freelancers	Para PYMES
Perfil profesional y portafolio	Explorar catálogo de servicios
Gestión de servicios publicados	Carrito de servicios y contratación
Solicitudes y contratos en curso	Gestión de contratos activos
Historial de trabajos	Historial de contrataciones
Sistema de mensajería interna	Sistema de mensajería interna
Calificaciones recibidas	Calificaciones enviadas y recibidas

7.5 Gestión de Perfil

7.5.1 Perfil de Freelancer

15. Ir al menú lateral y seleccionar "Mi Perfil".
16. Hacer clic en "Editar".
17. Modificar nombre, apellidos, cédula, teléfono y habilidades (separadas por coma).
18. Subir foto de perfil si se desea. La imagen se actualiza inmediatamente en el Sidebar.
19. Hacer clic en "Guardar cambios".

7.5.2 Perfil de PYME

20. Ir al menú lateral y seleccionar "Mi Perfil" → "Editar".
21. Completar datos de la empresa: nombre comercial, cédula jurídica, dirección, industria y tamaño.
22. Hacer clic en "Guardar cambios".

7.6 Publicación y Gestión de Servicios

7.6.1 Publicar un Servicio (Freelancer)

23. Ir a "Mis Servicios" → "Nuevo Servicio".
24. Completar: título, categoría, descripción detallada, precio en USD, días de entrega, revisiones incluidas e imágenes opcionales.
25. Hacer clic en "Guardar". El servicio quedará visible en el catálogo para todas las PYMES.

7.6.2 Explorar y Contratar un Servicio (PYME)

26. Ir a "Explorar Servicios" desde el menú principal.
27. Usar filtros: área/categoría, presupuesto mínimo y máximo, tiempo máximo de entrega, y ordenamiento.
28. Hacer clic en "Ver detalle" para ver información completa del servicio.
29. Para contratar directamente: "Contratar servicio". Para agrupar varios: "Agregar al carrito" y gestionar desde el Carrito.

7.7 Sistema de Contratación

7.7.1 Proceso para PYME

30. Enviar solicitud de contratación desde la página del servicio o desde el carrito.
31. El contrato queda en estado "Pendiente" hasta que el freelancer responda.
32. Hacer seguimiento: Pendiente → Aceptado → En Progreso → Finalizado.

7.7.2 Proceso para Freelancer

33. Revisar solicitudes recibidas en "Solicitudes".
34. Aceptar o rechazar la solicitud.
35. Mantener comunicación con la PYME durante el desarrollo del proyecto.
36. Marcar el trabajo como "Finalizado" cuando la entrega esté completa.

7.8 Sistema de Mensajería Interna

La plataforma incluye un sistema de chat organizado por contrato, con polling cada 5 segundos para mensajes en tiempo real.

Función	Descripción
Enviar mensajes	Chat en tiempo real dentro de cada contrato activo.
Historial de conversaciones	Lista de conversaciones ordenadas por actividad reciente.
Contador de no leídos	Badge numérico en cada conversación con mensajes sin leer.

7.9 Calificaciones y Reseñas

37. Ir a la sección "Calificaciones" en el menú lateral.
38. Seleccionar el contrato completado que se desea evaluar.
39. Otorgar puntuación del 1 al 5 estrellas y escribir un comentario.
40. Hacer clic en "Enviar calificación". Solo se permite una calificación por contrato.

Sistema de Reputación

- Las calificaciones son visibles en el perfil público de cada usuario.
- Solo se pueden calificar contratos con estado "Completado".
- Tanto PYMES como Freelancers pueden calificarse mutuamente.
- El promedio de estrellas aparece en las tarjetas del catálogo de servicios.

7.10 Historial de Trabajos

Cada usuario cuenta con un historial donde puede revisar toda su actividad dentro de la plataforma: contratos pasados, servicios realizados, reseñas recibidas y pagos asociados. Para acceder, ir a "Mis Contratos" o "Historial" en el menú lateral. Este registro ayuda a medir el desempeño y la trayectoria dentro de JobBridge.

8. CONCLUSIONES

El desarrollo de JobBridge representa un aporte significativo al ecosistema digital orientado al empleo, permitiendo la conexión efectiva entre freelancers y PYMES mediante un entorno web moderno, seguro y altamente funcional. Gracias a su arquitectura basada en tecnologías actuales como Node.js, React y MongoDB, la plataforma destaca por su escalabilidad, flexibilidad y capacidad de integrarse a diferentes procesos de contratación y gestión de servicios profesionales.

A nivel técnico, el sistema demuestra un uso adecuado de conceptos fundamentales como el diseño modular, la separación de responsabilidades, la protección de rutas mediante autenticación JWT, la comunicación cliente-servidor bajo estándares REST, y la integración de modelos de datos estructurados para la administración eficiente de usuarios, servicios, contratos, mensajes y calificaciones. Cada capa del sistema, desde los middlewares de seguridad hasta los contextos globales de React, fue diseñada con criterios técnicos sólidos que facilitan el mantenimiento y la incorporación de futuras mejoras.

Desde la perspectiva del usuario, JobBridge ofrece una experiencia clara, intuitiva y accesible, garantizando que tanto freelancers como PYMES puedan interactuar fácilmente con todas las funcionalidades de la plataforma, desde el registro hasta la gestión completa del ciclo de vida de los proyectos.

Logros Alcanzados

✓ Arquitectura monorepo cliente-servidor bien estructurada y documentada.

✓ Sistema de autenticación seguro con JWT, bcrypt y roles diferenciados.

✓ CRUD completo para servicios, contratos, usuarios y calificaciones.

✓ Middleware de seguridad: auth, role y upload con validaciones robustas.

✓ Mensajería interna por contrato con polling HTTP y auto-scroll.

✓ Sistema de calificaciones bidireccional con promedio visible en catálogo.

✓ Carrito de servicios para contrataciones múltiples con cálculo de tarifa.

✓ Interfaz responsiva con React 19, contextos globales y protección de rutas.

9. RECOMENDACIONES

9.1 Recomendaciones Técnicas

- Implementar pruebas automatizadas (unitarias con Jest y funcionales con Supertest) para garantizar la estabilidad del sistema ante futuras actualizaciones.
- Migrar el polling de mensajería a WebSockets (Socket.IO) para comunicación en tiempo real verdadera, eliminando solicitudes innecesarias cada 5 segundos.
- Integrar un sistema de logs avanzado (Winston o Pino) que facilite el monitoreo del comportamiento del servidor en producción.
- Configurar variables de entorno por ambiente con archivos .env.development y .env.production para separar configuraciones.
- Implementar control de versiones con ramas GitFlow: main, develop, feature/*. Configurar CI/CD básico con GitHub Actions.

9.2 Recomendaciones Funcionales

- Ampliar el sistema de notificaciones en tiempo real para alertar sobre nuevos mensajes, cambios en contratos o nuevas calificaciones.
- Incorporar un sistema de pago integrado (Stripe) para gestionar transacciones reales dentro de la plataforma.

9.3 Recomendaciones de Seguridad

- Implementar autenticación de dos factores (2FA) para reforzar la seguridad de las cuentas de usuario.
- Aplicar rate limiting en los endpoints de la API (express-rate-limit) para prevenir ataques de fuerza bruta.
- Restringir la configuración de CORS para producción, permitiendo solo el dominio real del frontend.
- Migrar JWT_SECRET a un gestor de secretos (AWS Secrets Manager o HashiCorp Vault) en entornos productivos.

9.4 Recomendaciones de Experiencia de Usuario

- Optimizar la plataforma para dispositivos móviles implementando diseño responsivo avanzado o Progressive Web App (PWA).
- Implementar modo oscuro para mayor accesibilidad y comodidad visual.
- Agregar un tour guiado interactivo para nuevos usuarios en su primer acceso a la plataforma.

10. BIBLIOGRAFÍA

Alura Latam. (s. f.). Node.JS: qué es, cómo funciona este entorno de ejecución de JavaScript. https://www.aluracursos.com/blog/node-js-que-es-como-funciona-este-entorno-de-ejecucion-de-javascript

Anthropic. (2026). *Claude* (claude-sonnet-4-6) [Large language model]. https://claude.ai

Arsys. (s. f.). Todo sobre la arquitectura cliente-servidor. https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor

Auth0. (2023). Introducción a los tokens web JSON. https://auth0.com/es/learn/json-web-tokens

AWS. (2023). ¿Qué es una interfaz de programación de aplicaciones (API)? Amazon Web Services, Inc. https://aws.amazon.com/es/what-is/api/

Cloudflare. (s. f.). ¿Qué es la seguridad de aplicaciones web? https://www.cloudflare.com/es-es/learning/security/what-is-web-application-security/

Gaurav, S. (2024). WebSockets and Socket.IO: Real-Time Communication with Node.js. DEV Community. https://dev.to/imsushant12/websockets-and-socketio-real-time-communication-with-nodejs-2j5f

Kenney, M., & Zysman, J. (2016). The rise of the platform economy. Issues in Science and Technology. https://issues.org/rise-platform-economy-big-data-work/

OpenAI. (2024). ChatGPT (versión del 13 de mayo). https://chat.openai.com/chat

Organización Internacional del Trabajo. (2021). World Employment and Social Outlook 2021: The role of digital labour platforms in transforming the world of work. OIT. https://www.ilo.org/publications/flagship-reports/

Red Hat. (s. f.). ¿Qué es una API? https://www.redhat.com/es/topics/api/what-are-application-programming-interfaces